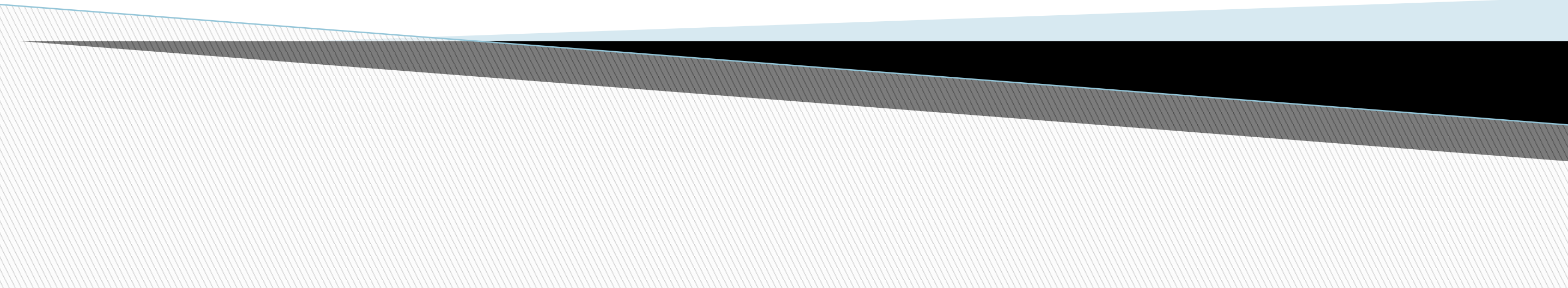


Process Synchronization



Important Note (1)

- Do not serialize your multi-process or multi-threaded code by placing **wait()** or **pthread_join()** to force an execution order.
 - Clarify with your teacher if you unable to get this?

```
int main()
{
    pthread_t th1;
    pthread_t th2;
    pthread_create(&th1, NULL, function, NULL);
    pthread_create(&th2, NULL, function, NULL);
    pthread_join(th1, NULL);
    pthread_join(th2, NULL);
}
```

- The above code only ordering guarantees here are that `pthread_join(th1, NULL);` will not return until thread 1 has exited and `pthread_join(th2, NULL);` will not return until thread 2 has exited. Consequently, the `main()` function will not return (and the process will not exit) until both thread 1 and thread 2 have exited.

6. Synchronization Tools

6.1 Background

6.2 The Critical-Section Problem

6.3 Peterson's Solution

6.4 Hardware Support for Synchronization

6.4.1 Memory Barriers

6.4.2 Hardware Instructions

6.4.3 Atomic Variables

6.5 Mutex Locks

6.6 Semaphores

6.6.1 Semaphore Usage

6.6.2 Semaphore Implementation

6.7 Monitors

6.8 Liveness

6.8.1 Deadlock

6.8.2 Priority Inversion

6.9 Evaluation

6.10 Summary

Concrete Theoretical understanding needed with textbook examples.

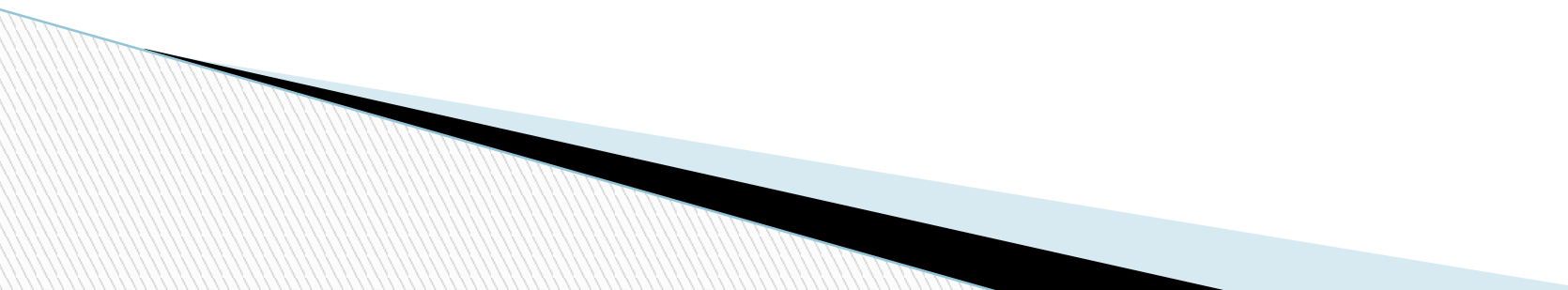
Student must have a basic understanding of these concepts

Concrete Theoretical understanding needed
+
Student should be able to write Pthread C code using Mutex Locks and Semaphores.

Multiple Processes

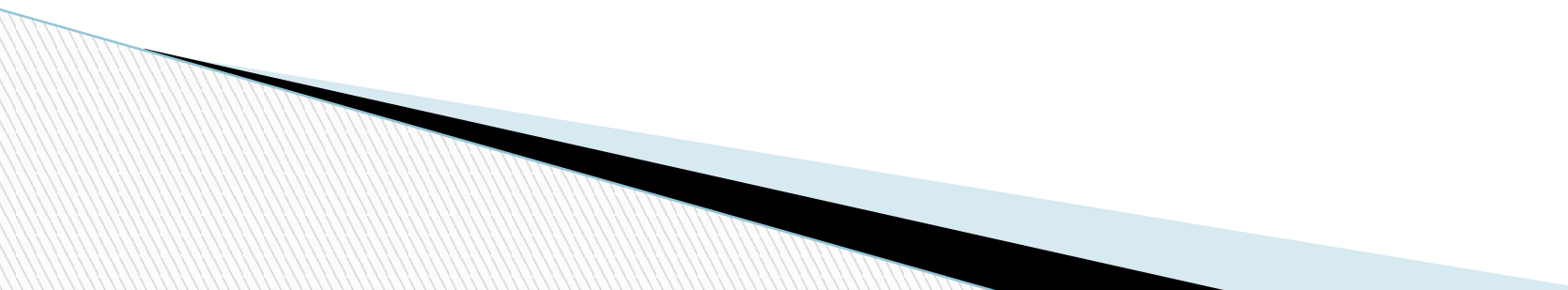
- Operating Systems is managing multiple processes:
 - Multiprogramming- The management of multiple processes within a uniprocessor system.
 - Multiprocessing- The management of multiple processes within a multiprocessor.
 - Distributed Processing- The management of multiple processes executing on multiple, distributed computer systems.e.g clusters
- Big Issue is Concurrency

Concurrency [1/2]

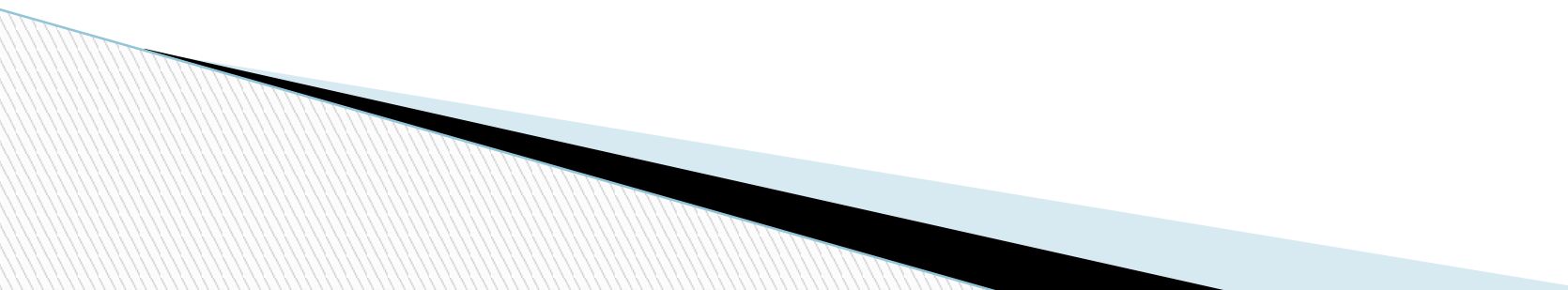
- Concurrency encompasses a host of design issues, including :
 - communication among processes
 - sharing of and competing for resources (such as memory, files, and I/O access)
 - synchronization of the activities of multiple processes
 - allocation of processor time to processes
- 

Concurrency [2/2]

Concurrency arises in:

- Multiple applications
 - Sharing time
 - Structured applications
 - Extension of modular design
 - Operating system structure
 - OS themselves implemented as a set of processes or threads
- 

Difficulties of Concurrency

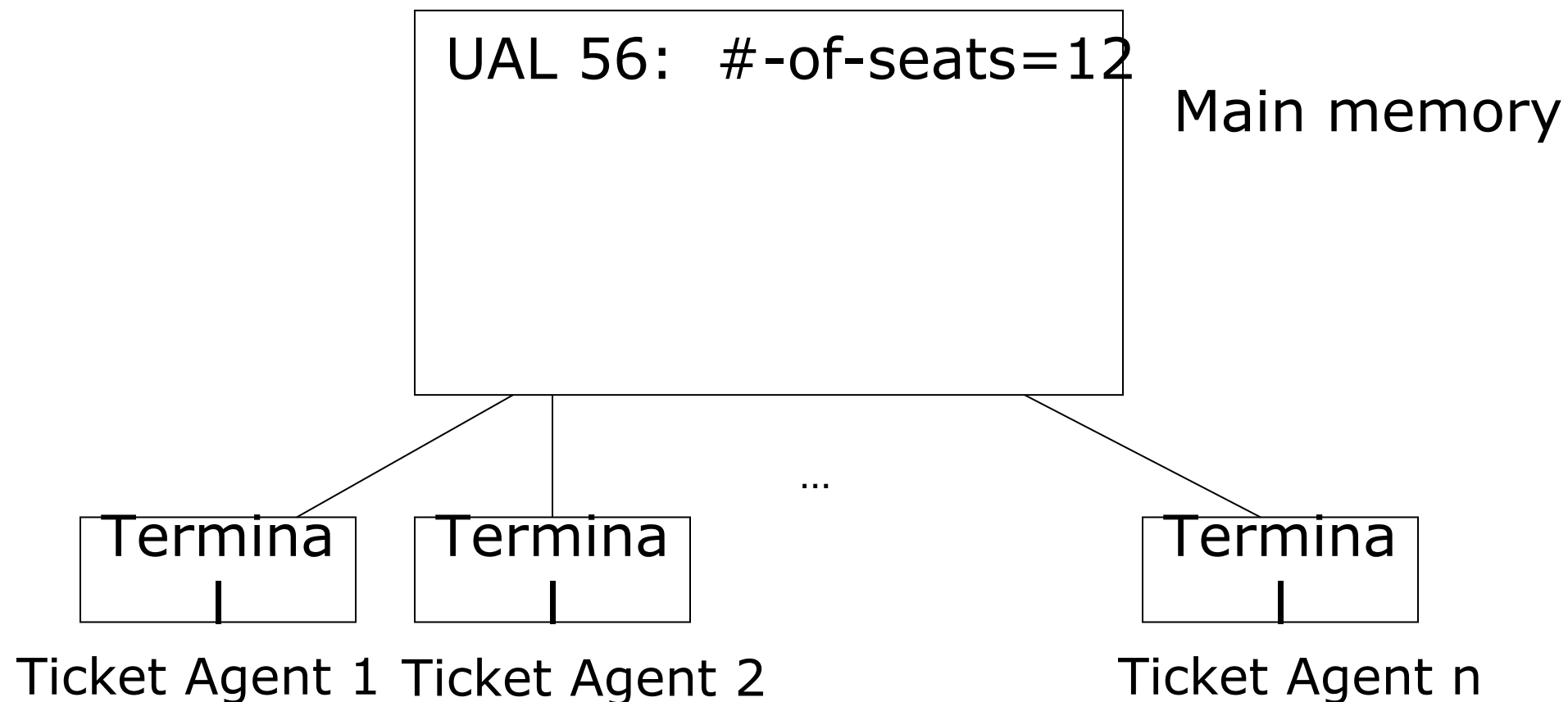
- Sharing of global resources
 - Writing a shared variable: the order of writes is important
 - Incomplete writes a major problem
 - Optimally managing the allocation of resources
 - Difficult to locate programming errors as results are not deterministic and reproducible.
- 

Race Condition

- A situation where several processes access and manipulate the same data concurrently and the outcome of the execution depends on the particular order in which the access takes place.

Example for Race condition

- Suppose a customer wants to book a seat on UAL 56. Ticket agent will check the #-of-seats. If it is greater than 0, he will grab a seat and decrement #-of-seats by 1.



Example for Race condition(cont.)

Ticket Agent 1

Ticket Agent 2

Ticket Agent 3

P1: LOAD #-of-seats

Q1: LOAD #-of-seats

R1: LOAD #-of-seats

P2: DEC 1

Q2: DEC 1

R2: DEC 1

P3: STORE #-of-seats

Q3: STORE #-of-seats

R3: STORE #-of-seats

Suppose instructions are interleaved as P1,Q1,R1,P2,Q2,R2,P3,Q3,R3

To solve the above problem, we must make sure that:

P1,P2,P3 must be completely executed before we execute Q1 or R1, or
Q1,Q2,Q3 must be completely executed before we execute P1 or R1, or
R1,R2,R3 must be completely executed before we execute P1 or Q1.

Race Condition

Producer

```
while (true) {  
    /* produce an item in next_produced */  
  
    while (count == BUFFER_SIZE)  
        ; /* do nothing */  
  
    buffer[in] = next_produced;  
    in = (in + 1) % BUFFER_SIZE;  
    count++;  
}
```

Consumer

```
while (true) {  
    while (count == 0)  
        ; /* do nothing */  
  
    next_consumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    count--;  
  
    /* consume the item in next_consumed */  
}
```

1. Producer Code:

- Produces item, waits if buffer full (`count == BUFFER_SIZE`).
- Writes item to `buffer[in]`, updates `in` (circular buffer), increments `count`.

2. Consumer Code:

- Waits if buffer empty (`count == 0`).
- Reads item from `buffer[out]`, updates `out`, decrements `count`.

Race Condition:

- Occurs because `count++` (producer) and `count--` (consumer) are **non-atomic** operations.
- Both threads may read/write `count` simultaneously, leading to incorrect buffer state.
- Example: Producer increments `count` but consumer reads stale `count`, causing buffer overflow/underflow.

Why?:

- No synchronization (e.g., mutex, semaphore) ensures mutual exclusion for shared variables (`count`, `buffer`, `in`, `out`).
- Interleaved execution of producer/consumer threads corrupts shared state.

Race Condition

A situation where several processes access and manipulate the same data concurrently and the outcome of the execution depends on the particular order in which the access takes place, is called a **race condition**.

```
while (true) {  
    /* produce an item in next_produced */  
  
    while (count == BUFFER_SIZE)  
        ; /* do nothing */  
  
    buffer[in] = next_produced;  
    in = (in + 1) % BUFFER_SIZE;  
    count++;  
}  
  
while (true) {  
    while (count == 0)  
        ; /* do nothing */  
  
    next_consumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    count--;  
  
    /* consume the item in next_consumed */  
}
```

$register_1 = count$
 $register_1 = register_1 + 1$
 $count = register_1$

$register_2 = count$
 $register_2 = register_2 - 1$
 $count = register_2$

Although the producer and consumer routines shown above are correct separately, they may not function correctly when executed concurrently. As an illustration, suppose that the value of the variable `count` is currently 5 and that the producer and consumer processes concurrently execute the statements “`count++`” and “`count--`”. Following the execution of these two statements, the value of the variable `count` may be 4, 5, or 6! The only correct result, though, is `count == 5`, which is generated correctly if the producer and consumer execute separately.

The Critical Section / Region Problem

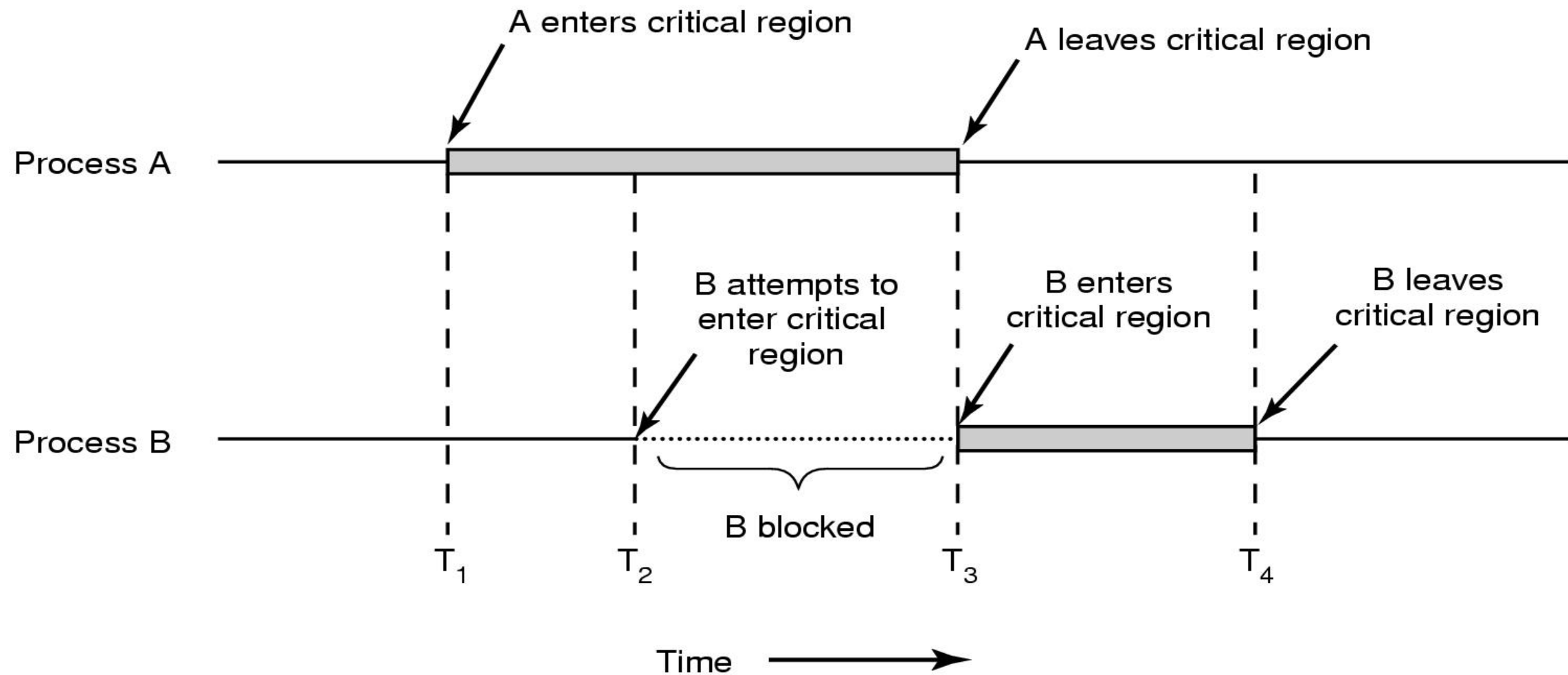
- Occurs in systems where multiple processes all compete for the use of shared data.
- Each process includes a section of code (the **critical section**) where it accesses this shared data.
- The problem is to ensure that **only one process at a time is allowed** to be operating in its critical section.

Critical Regions (1)

Conditions required to avoid race condition:

- No two processes may be simultaneously inside their critical regions.
- No assumptions may be made about speeds or the number of CPUs.
- No process should have to wait forever to enter its critical region.

Critical Regions (2)



Mutual exclusion using critical regions.

Critical Section Problem

P0/T0

```
while (true) {  
    entry section  
    critical section  
    exit section  
    remainder section  
}
```

P1/T1

```
while (true) {  
    entry section  
    critical section  
    exit section  
    remainder section  
}
```

How we solve CS Problem in Code?

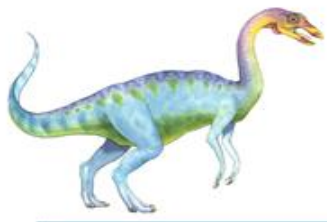
- **Mutex:** Exclusive access to a resource.
- **Semaphore:** Manages access to a pool of resources or signals.
- **Condition Variable:** Coordinates threads based on conditions, always paired with a mutex.



Solution to Critical-Section Problem

1. **Mutual Exclusion** - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections
2. **Progress** - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely
3. **Bounded Waiting** - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted
 - Assume that each process executes at a nonzero speed
 - No assumption concerning **relative speed** of the n processes





Critical-Section Handling in OS

Two approaches depending on if kernel is preemptive or non-preemptive

- **Preemptive** – allows preemption of process when running in kernel mode
- **Non-preemptive** – runs until exits kernel mode, blocks, or voluntarily yields CPU
 - ▶ Essentially free of race conditions in kernel mode

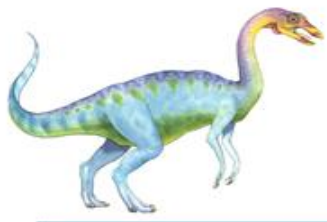




Peterson's Solution

- Good algorithmic description of solving the problem
- Two process solution
- Assume that the `load` and `store` machine-language instructions are atomic; that is, cannot be interrupted
- The two processes share two variables:
 - `int turn;`
 - `Boolean flag[2]`
- The variable `turn` indicates whose turn it is to enter the critical section
- The `flag` array is used to indicate if a process is ready to enter the critical section. `flag[i] = true` implies that process P_i is ready!





Algorithm for Process P_i

```
do {  
    flag[i] = true;  
    turn = j;  
    while (flag[j] && turn == j);  
        critical section  
    flag[i] = false;  
        remainder section  
} while (true);
```





Peterson's Solution (Cont.)

- Provable that the three CS requirement are met:
 1. Mutual exclusion is preserved
 - P_i enters CS only if:
 - either `flag[j] = false` or `turn = i`
 2. Progress requirement is satisfied
 3. Bounded-waiting requirement is met





Synchronization Hardware

- Many systems provide hardware support for implementing the critical section code.
- All solutions below based on idea of **locking**
 - Protecting critical regions via locks
- Uniprocessors – could disable interrupts
 - Currently running code would execute without preemption
 - Generally too inefficient on multiprocessor systems
 - ▶ Operating systems using this not broadly scalable
- Modern machines provide special atomic hardware instructions
 - ▶ **Atomic** = non-interruptible
 - Either test memory word and set value
 - Or swap contents of two memory words





test_and_set Instruction

Definition:

```
boolean test_and_set (boolean *target)
{
    boolean rv = *target;
    *target = TRUE;
    return rv;
}
```

1. Executed atomically
2. Returns the original value of passed parameter
3. Set the new value of passed parameter to “TRUE”.





Solution using test_and_set()

- Shared Boolean variable lock, initialized to FALSE
- Solution:

```
do {  
    while (test_and_set(&lock))  
        ; /* do nothing */  
        /* critical section */  
    lock = false;  
        /* remainder section */  
} while (true);
```



Busy Waiting

Busy waiting is a synchronization technique where a thread repeatedly checks (spins) for a condition to become true, consuming CPU cycles while waiting. Here's a compact technical breakdown:

Why Busy Waiting?

- **Low Latency:** Avoids the overhead of context switching or sleeping, making it suitable for very short waits.
 - **Simplicity:** Easy to implement for simple synchronization tasks.
-

How It Works

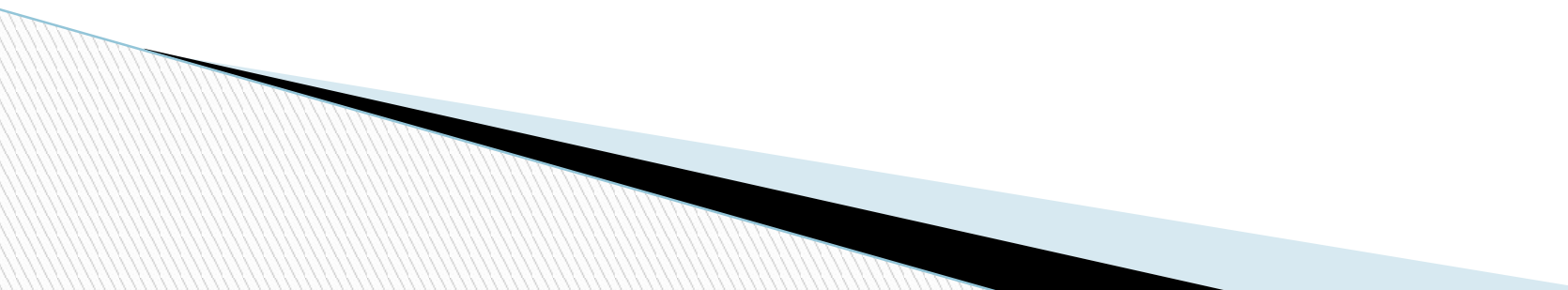
1. **Spin Loop:** The thread continuously checks a condition (e.g., a flag or lock) in a loop:

```
while (!condition) {} // Busy wait
```

2. **No Yielding:** The thread does not yield the CPU, keeping the core busy.

Busy Waiting

Technical Details:

- CPU Usage: Consumes 100% CPU on the waiting core, wasting resources.
 - No OS Involvement: Does not involve the OS scheduler, reducing latency for very short waits.
 - Starvation Risk: If the condition is never met, the thread spins indefinitely.
- 

Busy Waiting

Use cases:

- Short Waits: Ideal for very short critical sections or low-contention scenarios.
- Real-Time Systems: Used in real-time systems where predictable latency is critical.
- Spinlocks: Common in kernel-level programming for lightweight synchronization.

Alternatives:

- Blocking Waits: Use OS primitives like `pthread_cond_wait` or semaphores to sleep the thread, freeing CPU resources.

Mutex Locks

- OS designers build software tools to solve critical section problem
- Simplest is mutex lock
- Protect critical regions with it by first **acquire()** a lock then **release()** it
 - Boolean variable indicating if lock is available or not
- Calls to **acquire()** and **release()** must be atomic
 - Usually implemented via hardware atomic instructions
- But this solution requires **busy waiting**
- **Busy Waiting means** busy-looping or spinning is a technique in which a process repeatedly checks to see if a condition is true, such as whether keyboard input or a lock is available
 - Mutex lock therefore called a **spinlock**

Pthreads Synchronization – Mutex Lock

```
#include <pthread.h>
```

```
Pthread_mutex_t mutex;
```

```
/* create the mutex lock */
```

```
Pthread_mutex_init(&mutex, NULL);
```

1st Arg: Mutex initializer

2nd Arg: Attributes, NULL means no error checks will be performed

Pthreads Synchronization – Mutex Lock

- The mutex is acquired and released with the `pthread_mutex_lock()`
- and `pthread_mutex_unlock()` functions.
- If the mutex lock is unavailable when `pthread_mutex_lock()` is invoked, the calling thread is blocked until the owner invokes `pthread_mutex_unlock()`.

What is a Semaphore

- a **semaphore** is a variable or abstract data type used to control access to a common resource by multiple processes in a concurrent **system** such as a multiprogramming **operating system**.

Semaphore

- Synchronization tool that does not require busy waiting
- Semaphore S – integer variable
- Two standard operations modify S : **wait()** and **signal()**
- Less complicated
- Can only be accessed via two indivisible (atomic) operations

Definition of wait() operation:

```
wait (S) {  
    while (S <= 0)  
        ; // busy wait  
    S--;  
}
```

Definition of signal() operation:

```
signal (S) {  
    S++;  
}
```

- when one process modifies the semaphore value, no other process can simultaneously modify that same semaphore value.

Semaphore Usage

Counting Semaphore:

- Semaphores which allow an arbitrary resource count are called **counting semaphores**,
- Counting semaphores can be used to control access to a given resource consisting of a finite number of instances
- Can implement a counting semaphore S as a binary semaphore

Binary Semaphore:

- while semaphores which are restricted to the values 0 and 1 (or locked/unlocked, unavailable/available) are called **binary semaphores** and are used to implement locks.
- Binary semaphores behave similarly to mutex locks.
- Can implement a counting semaphore S as a binary semaphore

Semaphore Usage

- consider two concurrently running processes: $P1$ with a statement $S1$ and $P2$ with a statement $S2$.
- It is required that $S2$ be executed only after $S1$ has completed. We can implement this scheme readily by letting $P1$ and $P2$ share a common semaphore synch, initialized to 0.

P1:

S_1 ;

signal(synch);

P2:

wait(synch);

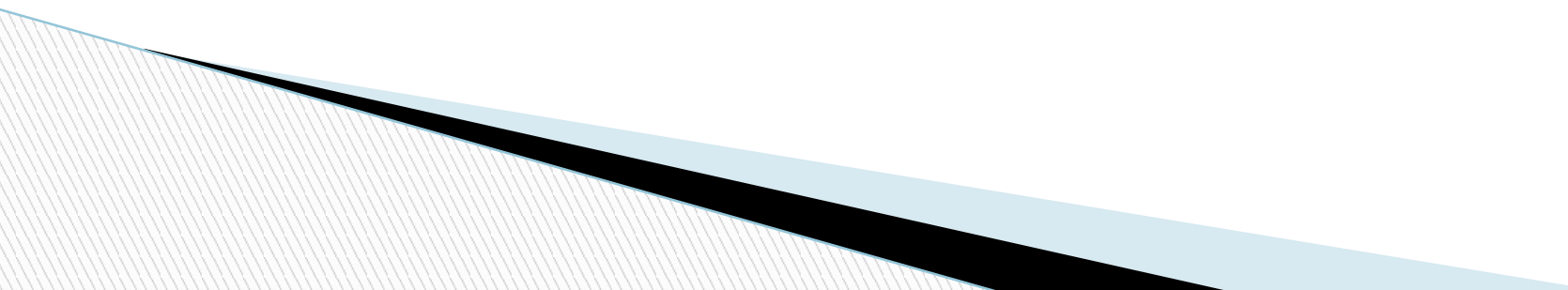
S_2 ;

- Because synch is initialized to 0, $P2$ will execute $S2$ only after $P1$ has invoked signal(synch), which is after statement $S1$ has been executed.

Semaphore Implementation

- Must guarantee that no two processes can execute **wait()** and **signal()** on the same semaphore at the same time
- Thus, implementation becomes the critical section problem where the wait and signal code are placed in the critical section
 - Could now have **busy waiting** in critical section implementation
- Note that applications may spend lots of time in critical sections and therefore this is not a good solution

Semaphore Implementation with no Busy waiting

- With each semaphore there is an associated waiting queue
 - Two operations:
 - **block** – place the process invoking the operation on the appropriate waiting queue
 - **wakeup** – remove one of processes in the waiting queue and place it in the ready queue
- 



Implementation with no Busy waiting (Cont.)

```
wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();
    }
}

signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);
    }
}
```



POSIX - Semaphores

```
#include <semaphore.h>
```

```
Sem_t sem;
```

```
/* Create the semaphore and initialize it to 1 */
```

```
Sem_init(&sem, 0, 1);
```

The `sem_init()` function is passed three parameters:

1. A pointer to the semaphore
2. A flag indicating the level of sharing
3. The semaphore's initial value

POSIX - Semaphores

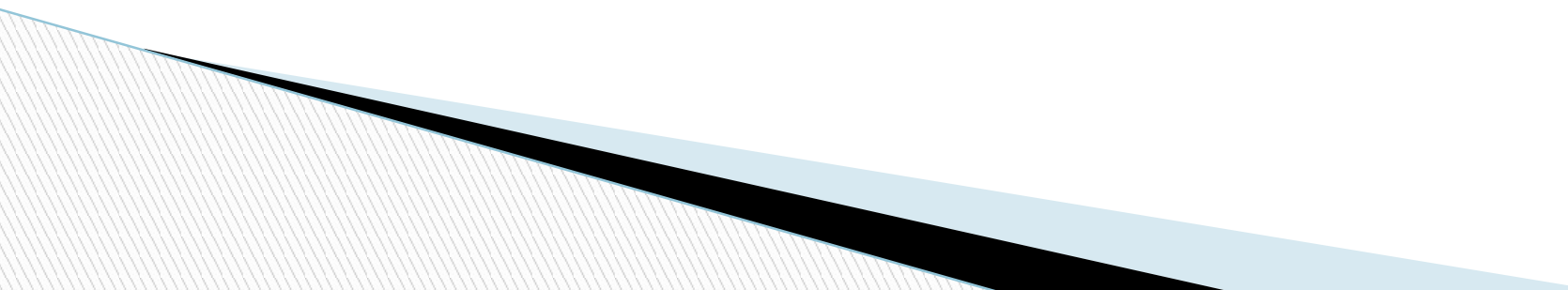
```
/* acquire the semaphore */
```

```
Sem_wait(&sem);
```

```
/* critical section */
```

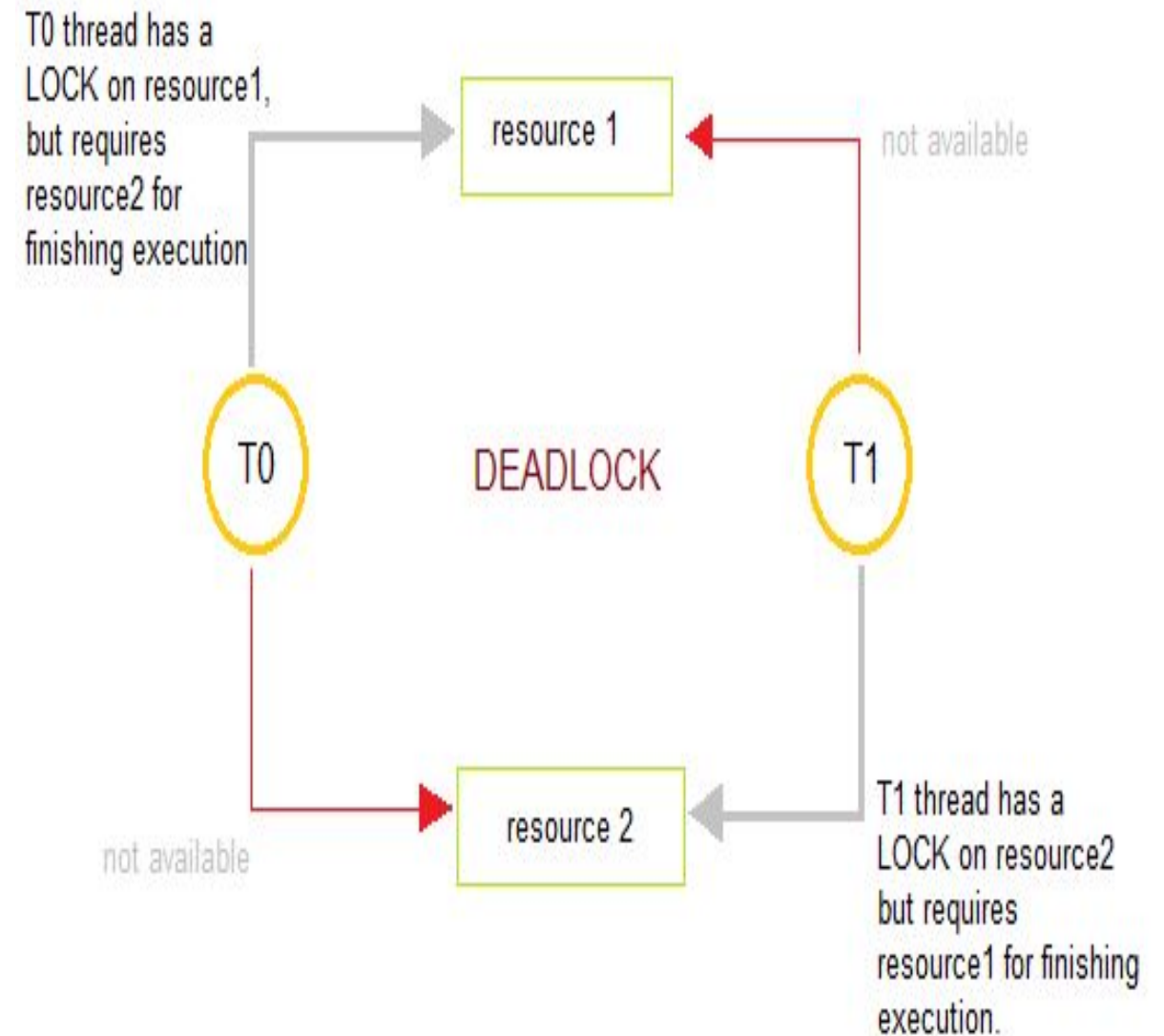
```
/* release the semaphore */
```

```
Sem_post(&sem);
```



Deadlock

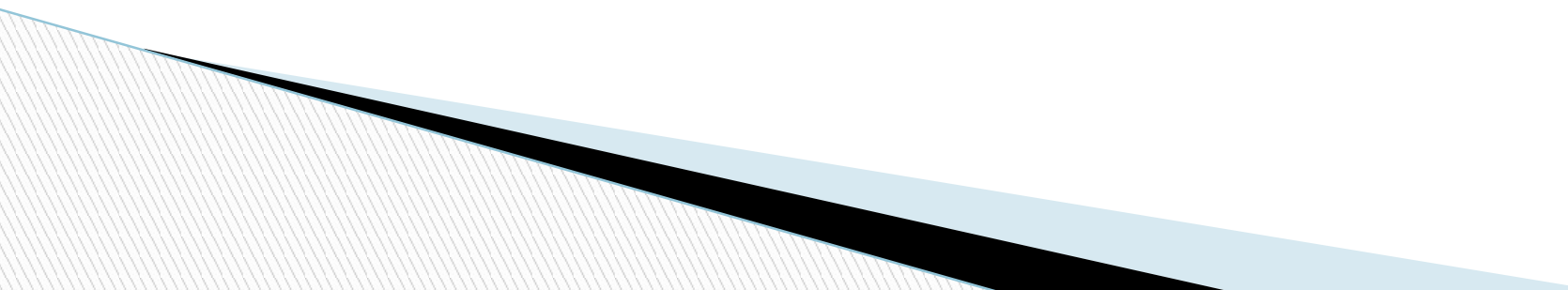
- Deadlocks are a set of blocked processes each holding a resource and waiting to acquire a resource held by another process



Starvation

- **Starvation** is the name given to the indefinite postponement of a process because it requires some resource before it can run, but the resource, though available for allocation, is never allocated to this process.

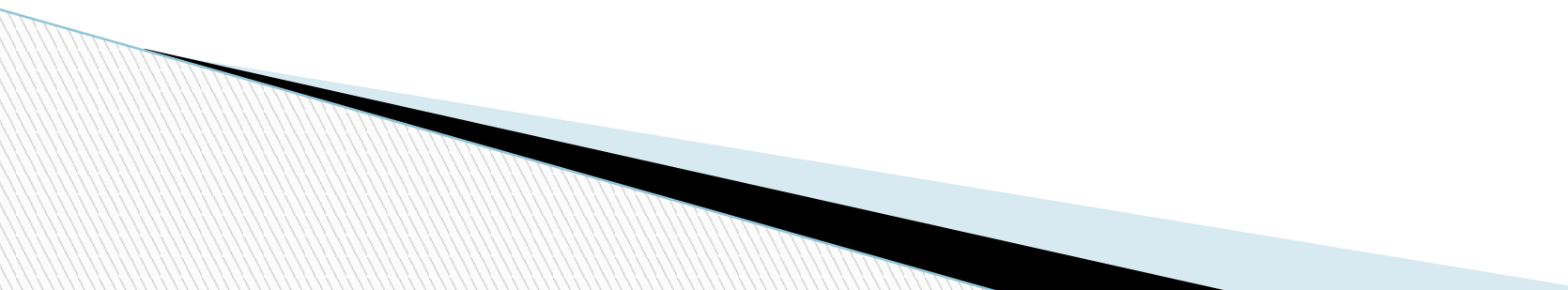
Deadlock Vs. Starvation

- ❑ Deadlock refers to the situation when processes are stuck in circular waiting for the resources.
 - ❑ On the other hand, starvation occurs when a process waits for a resource indefinitely.
- 

Priority Inversion

- **Priority Inversion** – Scheduling problem when lower-priority process holds a lock needed by higher-priority process
 - Solved via **priority-inheritance protocol**

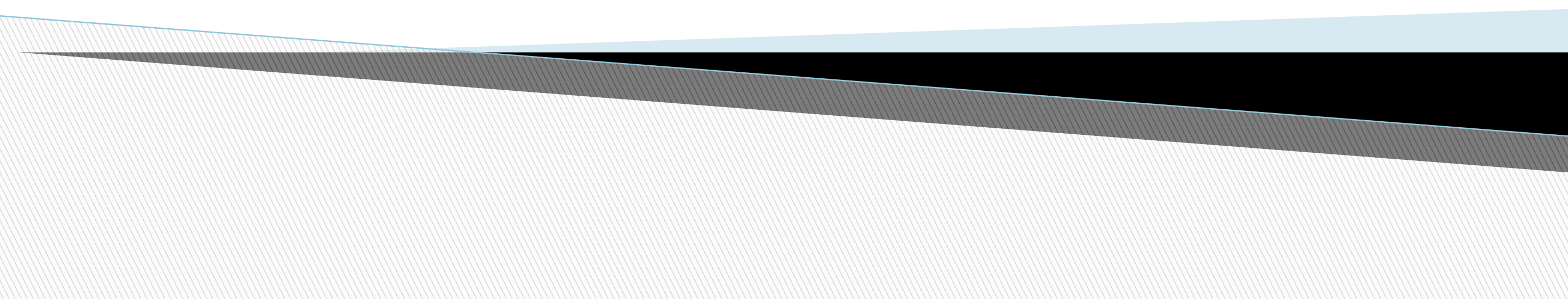
Priority Inheritance Protocol

- when a job blocks one or more high-priority jobs, it ignores its original priority assignment and executes its critical section at an elevated priority level.
 - After executing its critical section and releasing its locks, the process returns to its original priority level
- 

6.8 Liveness

- Liveness is a property of concurrent systems ensuring that desired events or actions eventually occur, despite delays or interference from other threads. It guarantees progress and prevents scenarios like:
 - **Deadlock:** Threads block indefinitely, each holding a resource needed by another.
 - **Starvation:** A thread is perpetually denied resources, preventing progress.
 - **Livelock:** Threads keep retrying an operation but make no progress due to repeated interference.

Chapter 7: Synchronization Examples



7. Synchronization Examples

- 7.1 Classic Problems of Synchronization
 - 7.1.1 The Bounded-Buffer Problem
 - 7.1.2 The Readers-Writers Problem
 - 7.1.3 The Dining-Philosophers Problem

Textbook pseudo code
+
Python code examples

~~7.2 Synchronization within the Kernel~~

- 7.3 POSIX Synchronization
 - 7.3.1 POSIX Mutex Locks
 - 7.3.2 POSIX Semaphores
 - 7.3.3 POSIX Condition Variables

linked

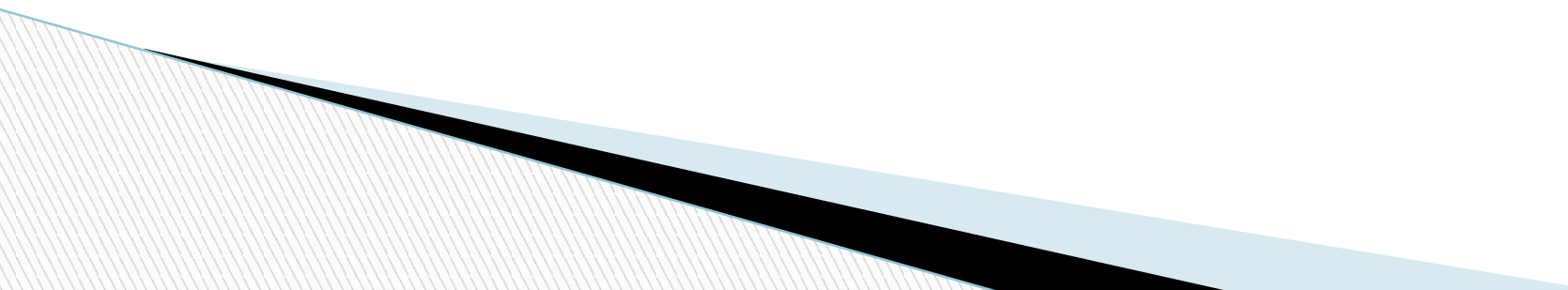
~~7.4 Synchronization in Java~~

~~7.5 Alternative Approaches~~

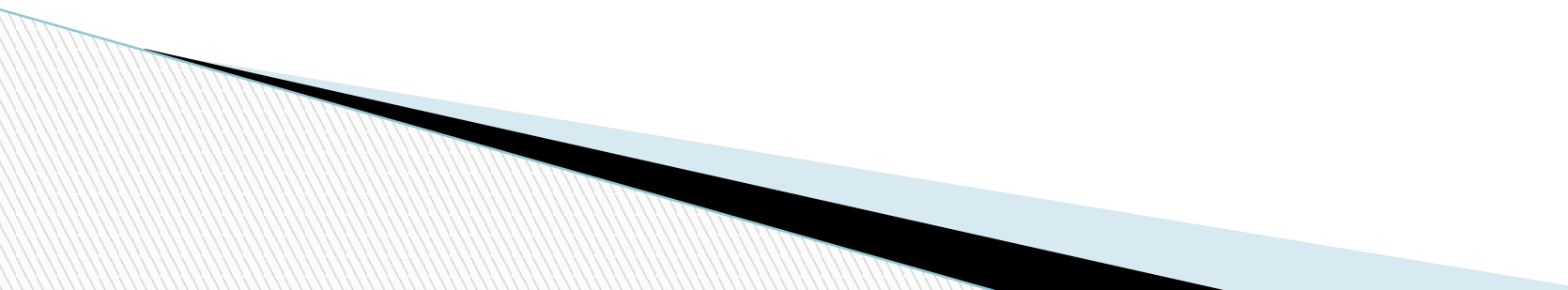
7.6 Summary

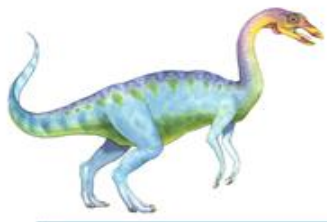
Practice Exercises

Outline

- Explain the bounded-buffer synchronization problem
 - Explain the readers-writers synchronization problem
 - Explain and dining-philosophers synchronization problems
 - Describe the tools used by Linux and Windows to solve synchronization problems.
 - Illustrate how POSIX and Java can be used to solve process synchronization problems
- 

Classical Problems of Synchronization

- Classical problems used to test newly-proposed synchronization schemes
 - Bounded Buffer Problem
 - Dining-Philosophers Problem
 - Readers and Writers Problem
- 



Bounded-Buffer Problem

- n buffers, each can hold one item
- Semaphore `mutex` initialized to the value 1
- Semaphore `full` initialized to the value 0
- Semaphore `empty` initialized to the value n





Bounded Buffer Problem (Cont.)

- The structure of the producer process

```
do {  
    ...  
    /* produce an item in next_produced */  
    ...  
    wait(empty) ;  
    wait(mutex) ;  
    ...  
    /* add next produced to the buffer */  
    ...  
    signal(mutex) ;  
    signal(full) ;  
} while (true) ;
```





Bounded Buffer Problem (Cont.)

- The structure of the consumer process

```
Do {  
    wait(full) ;  
    wait(mutex) ;  
    ...  
    /* remove an item from buffer to next_consumed */  
    ...  
    signal(mutex) ;  
    signal(empty) ;  
    ...  
    /* consume the item in next consumed */  
    ...  
} while (true) ;
```



Readers-Writers Problem

- A data set is shared among a number of concurrent processes
 - **Readers** – only read the data set; they do *not* perform any updates
 - **Writers** – can both read and write
- Problem – allow multiple readers to read at the same time
 - Only one single writer can access the shared data at the same time
- Several variations of how readers and writers are considered – all involve some form of priorities

Readers-Writers Problem (Cont.)

□ Shared Data

- Data set
- Semaphore `rw_mutex` initialized to 1
- Semaphore `mutex` initialized to 1
- Integer `read_count` initialized to 0

Readers-Writers Problem (Cont.)

- The structure of a writer process

```
while (true) {  
    wait(rw_mutex) ;  
    ...  
    /* writing is performed */  
    ...  
    signal(rw_mutex) ;  
}
```

Readers-Writers Problem (Cont.)

□ The structure of a reader process

```
while (true){
    wait(mutex);
    read_count++;
    if (read_count == 1) /* first reader */
        wait(rw_mutex);
        signal(mutex);

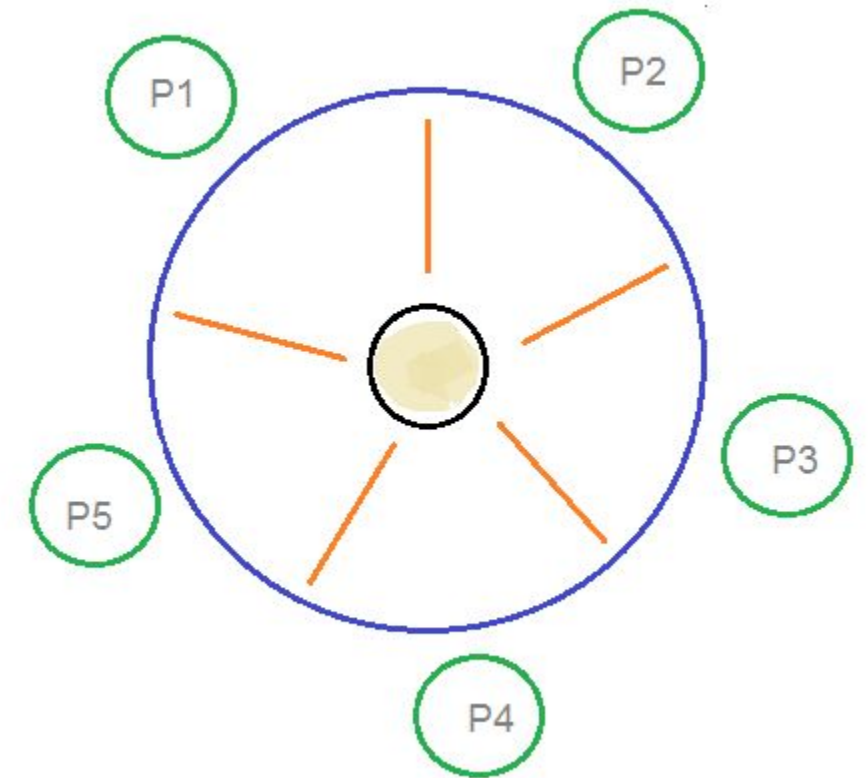
    ...
    /* reading is performed */
    ...
    wait(mutex);
    read_count--;
    if (read_count == 0) /* last reader */
        signal(rw_mutex);
    signal(mutex);
}
```

Readers-Writers Problem Variations

- The solution in previous slide can result in a situation where a writer process never writes. It is referred to as the “First reader-writer” problem.
- The “Second reader-writer” problem is a variation the first reader-writer problem that state:
 - Once a writer is ready to write, no “newly arrived reader” is allowed to read.
- Both the first and second may result in starvation. leading to even more variations
- Problem is solved on some systems by kernel providing reader-writer locks

Dining-Philosophers Problem

- Consider there are five philosophers sitting around a circular dining table. The dining table has five chopsticks and a bowl of rice.



Dining-Philosophers Solution

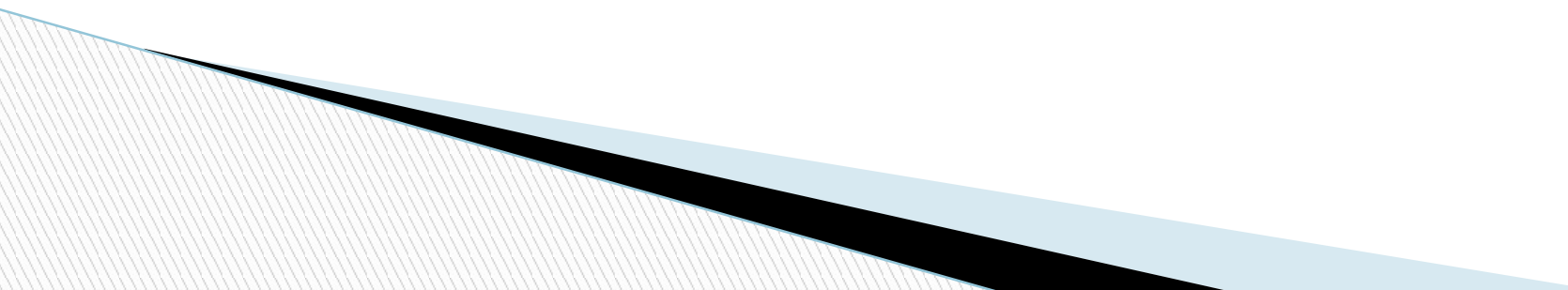
- it is clear that a philosopher can think for an indefinite amount of time. But when a philosopher starts eating, he has to stop at some point of time

Dining-Philosophers Solution 1

- An array of five semaphores, **stick[5]**, for each of the five chopsticks.
- The code for each philosopher looks like:

```
while(TRUE) {  
    wait(stick[i]);  
    wait(stick[(i+1) % 5]); // mod is used because if i=5, next  
                           // chopstick is 1 (dining table is circular)  
  
    /* eat */  
    signal(stick[i]);  
    signal(stick[(i+1) % 5]);  
    /* think */  
}
```

Dining-Philosophers Solution 1

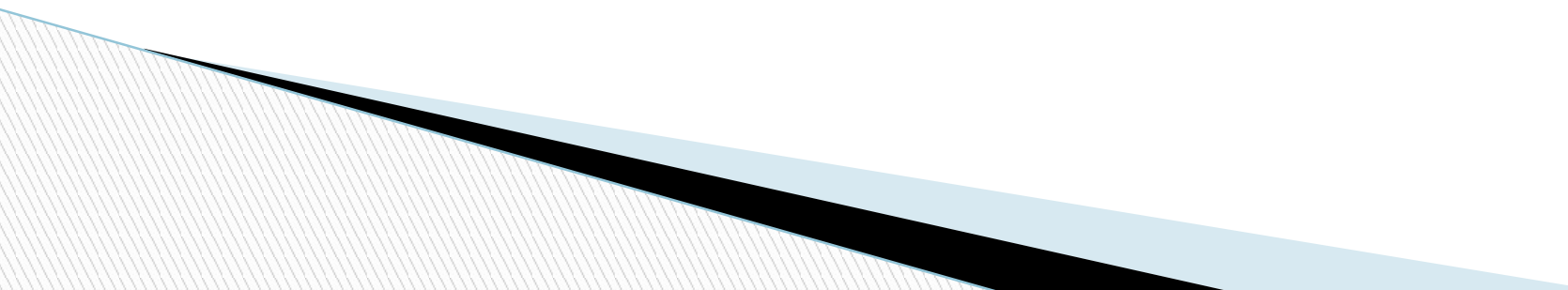
- When a philosopher wants to eat the rice, he will wait for the chopstick at his left and picks up that chopstick.
 - Then he waits for the right chopstick to be available, and then picks it too. After eating, he puts both the chopsticks down
- 

Dining-Philosophers Problem

- But if all five philosophers are hungry simultaneously, and each of them pickup one chopstick, then a deadlock situation occurs:

Deadlock implies starvation but starvation does not imply deadlock

Dining-Philosophers Solution 2

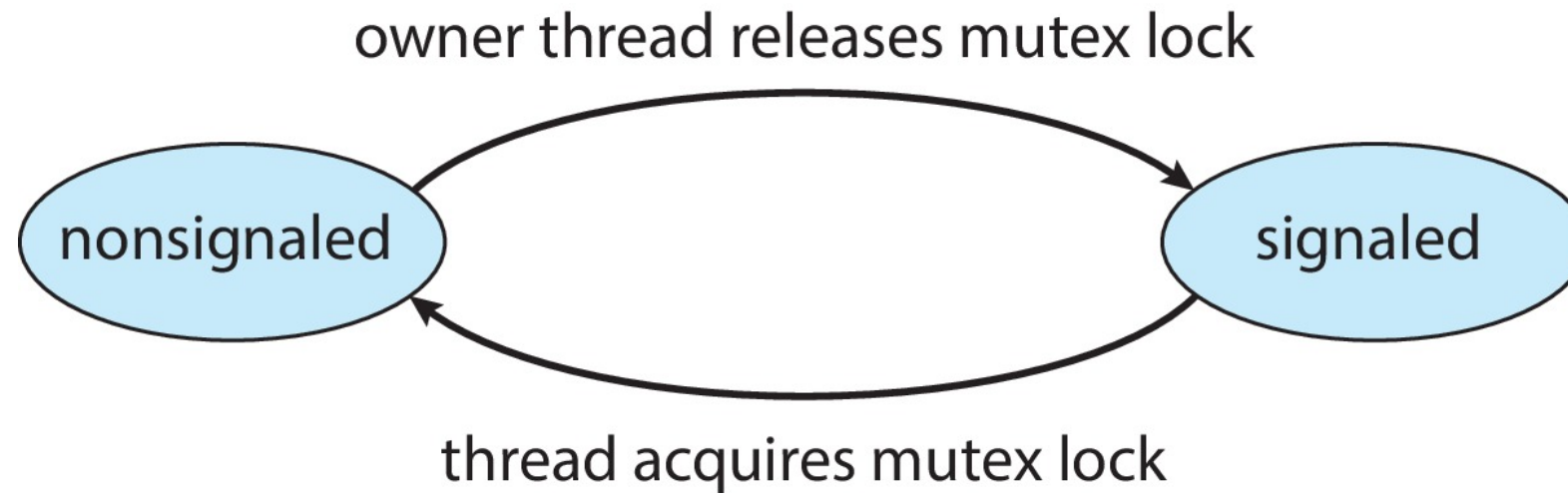
- Two Possible solutions are :
 - A philosopher must be allowed to pick up the chopsticks only if both the left and right chopsticks are available.
 - Allow only four philosophers to sit at the table. That way, if all the four philosophers pick up four chopsticks, there will be one chopstick left on the table. So, one philosopher can start eating and eventually, two chopsticks will be available. In this way, deadlocks can be avoided.
- 

Kernel Synchronization - Windows

- Uses interrupt masks to protect access to global resources on uniprocessor systems
- Uses **spinlocks** on multiprocessor systems
 - Spinlocking-thread will never be preempted
- Also provides **dispatcher objects** user-land which may act mutexes, semaphores, events, and timers
 - **Events**
 - An event acts much like a condition variable
 - Timers notify one or more thread when time expired
 - Dispatcher objects either **signaled-state** (object available) or **non-signaled state** (thread will block)

Kernel Synchronization - Windows

□ Mutex dispatcher object



Linux Synchronization

- Linux:
 - Prior to kernel Version 2.6, disables interrupts to implement short critical sections
 - Version 2.6 and later, fully preemptive
- Linux provides:
 - Semaphores
 - Atomic integers
 - Spinlocks
 - Reader-writer versions of both
- On single-CPU system, spinlocks replaced by enabling and disabling kernel preemption

Linux Synchronization

□ Atomic variables

atomic_t is the type for atomic integer

□ Consider the variables

```
atomic_t counter;  
int value;
```

<i>Atomic Operation</i>	<i>Effect</i>
<code>atomic_set(&counter,5);</code>	<code>counter = 5</code>
<code>atomic_add(10,&counter);</code>	<code>counter = counter + 10</code>
<code>atomic_sub(4,&counter);</code>	<code>counter = counter - 4</code>
<code>atomic_inc(&counter);</code>	<code>counter = counter + 1</code>
<code>value = atomic_read(&counter);</code>	<code>value = 12</code>

POSIX Synchronization

- POSIX API provides
 - mutex locks
 - semaphores
 - condition variable
- Widely used on UNIX, Linux, and macOS

POSIX Mutex Locks

□ Creating and initializing the lock

```
#include <pthread.h>

pthread_mutex_t mutex;

/* create and initialize the mutex lock */
pthread_mutex_init(&mutex, NULL);
```

□ Acquiring and releasing the lock

```
/* acquire the mutex lock */
pthread_mutex_lock(&mutex);

/* critical section */

/* release the mutex lock */
pthread_mutex_unlock(&mutex);
```

POSIX Semaphores

- POSIX provides two versions – **named** and **unnamed**.
- Named semaphores can be used by unrelated processes, unnamed cannot.

POSIX Named Semaphores

- Creating and initializing the semaphore:

```
#include <semaphore.h>
sem_t *sem;

/* Create the semaphore and initialize it to 1 */
sem = sem_open("SEM", O_CREAT, 0666, 1);
```

- Another process can access the semaphore by referring to its name **SEM**.
- Acquiring and releasing the semaphore:

```
/* acquire the semaphore */
sem_wait(sem);

/* critical section */

/* release the semaphore */
sem_post(sem);
```

POSIX Unnamed Semaphores

□ Creating

```
#include <semaphore.h>
sem_t sem;

/* Create the semaphore and initialize it to 1 */
sem_init(&sem, 0, 1);
```

□ Acquiring and releasing the semaphore:

```
/* acquire the semaphore */
sem_wait(&sem);

/* critical section */

/* release the semaphore */
sem_post(&sem);
```

POSIX Condition Variables

- Since POSIX is typically used in C/C++ and these languages do not provide a monitor, POSIX condition variables are associated with a POSIX mutex lock to provide mutual exclusion: Creating and initializing the condition variable:

```
pthread_mutex_t mutex;  
pthread_cond_t cond_var;  
  
pthread_mutex_init(&mutex, NULL);  
pthread_cond_init(&cond_var, NULL);
```

POSIX Condition Variables

- Thread waiting for the condition **a == b** to become true:

```
pthread_mutex_lock(&mutex);  
while (a != b)  
    pthread_cond_wait(&cond_var, &mutex);  
  
pthread_mutex_unlock(&mutex);
```

- Thread signaling another thread waiting on the condition variable:

```
pthread_mutex_lock(&mutex);  
a = b;  
pthread_cond_signal(&cond_var);  
pthread_mutex_unlock(&mutex);
```